

C++

1 C++语言入门

- 1.1 第一个C++程序
- 1.2 C++程序的注释
- 1.3 变量
- 1.4 常量
- 1.5 关键字
- 1.6 标识符命名规则

2 数据类型

- 2.1 整型
- 2.2 sizeof 关键字
- 2.3 实型 (浮点型)
- 2.4 字符型
- 2.5 转义字符
- 2.6 字符串型
- 2.7 布尔类型(bool)
- 2.8 数据的输入

3 运算符

- 3.1 算术运算符
- 3.2 赋值运算符
- 3.3 比较运算符
- 3.4 逻辑运算符
 - 3.4.1 **逻辑非**
 - 3.4.2 逻辑与
 - 3.4.3 逻辑或

4 程序流程结构

- 4.1 选择结构
 - 4.1.1 if语句
 - 4.1.2 三目运算符
 - 4.1.3 switch语句
- 4.2 循环结构
 - 4.2.1 while循环语句
 - 4.2.2 do...while循环
 - 4.2.3 for 循环
 - 4.2.4 嵌套循环
- 4.3 跳转语句
 - 4.3.1 break 语句
 - 4.3.2 continue 语句
 - 4.3.3 goto 语句

5 数组

- 5.1 概述
- 5.2 一维数组
 - 5.2.1 一维数组的定义方式:
 - 5.2.2 一维数组数组名
 - 5.2.3 冒泡排序
- 5.3 二维数组
 - 5.3.1 **二维数组的定义方式:**
 - 5.3.2 二维数组的赋值方式
 - 5.3.3 二维数组数组名
 - 5.3.3 二维数组应用案例
 - 5.3.4 二维数组排序

6 函数

- 6.1 概述
- 6.2 函数的定义

- 6.3 函数的调用
- 6.4 值传递
- 6.5 函数的常见样式
- 6.6 函数的声明
- 6.7 函数的分文件填写
- 7 指针
 - 7.1 指针的基本概念
 - 7.2 指针的定义和操控
 - 7.3 指针所占的内存空间
 - 7.4 空指针与野指针
 - 7.5 const修饰指针
 - 7.6 数组与指针的关系[原创]
 - 指针数组与数组指针
- 8 结构体
 - 8.1 结构体的定义
 - 8.2 结构体数组
 - 8.3 数组结构体[原创]
 - 8.4 结构体的构造函数
- 链表
 - 单向链表与双向链表的创建
 - 向链表里写入与删去数据
 - 在头部插入数据
 - 在某一个pos值上插入元素
 - 在尾部插入元素**
 - 删除某一结点
- 9 类

C++

C++语言是一门广泛被使用的语言，学习C语言，可以使我们更加深入的了解到编程语言的运行方式和底层逻辑，下面，让我们从零开始，学习C++

1 C++语言入门

1.1 第一个C++程序

我们利用C++输入 hello world

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "hello world"<< endl;
7
8      system("pause");
9
10     return(0);
11 }
```

1.2 C++程序的注释

1. 单行注释

- 使用 `//` 来注释一行代码

2. 多行注释

- 使用 `/* ..... */` 来多行注释

3. 例子

```
1  #include <iostream>
2
3  /* 这是一个头文件
4   用于给定指定函数名称*/
5
6  using namespace std;
7  int main()
8  {
9      cout << "hello world"<< endl; //输出hello world
10
11     system("pause");
12
13     return(0);
14 }
```

1.3 变量

- **作用：**给定一段指定的内存空间取名以方便我们操纵这段内存
- **语法：**`数据类型 变量名 = 初始值 (int a =10)`
- 实例

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int a = 10;
6      cout<<"a="<<a<<endl;
7      system("pause");
8      return(0);
9  }
```

此时C++会输出“a=10”

1.4 常量

作用：用于记录程序中不可更改的数据

C++有两种定义常量的方法

1. **#define 宏常量：** `#define 常量名 = 常量值`
 - 通常在代码文件上方定义,表示一个常量

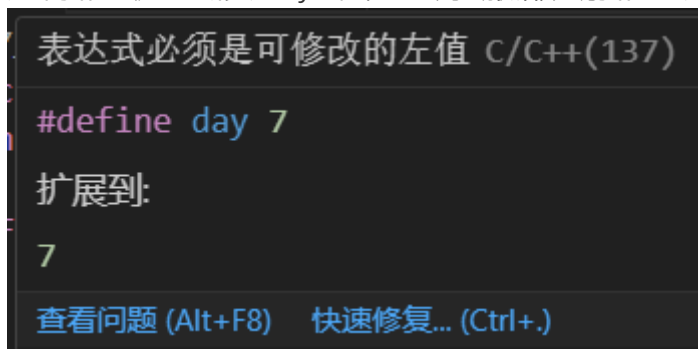
2. **const**修饰的变量: `const 数据类型 常量名 =常量值`

- 通常`在变量定义加关键字const`修饰该变量为常量,不可更改

示例1:

```
1 #include <iostream>
2 using namespace std;
3
4 #define day 7
5 int main()
6 {
7     cout<<"一周有"<<day<<"天"<<endl;
8     system("pause");
9     return(0);
10 }
```

- 如果我们强加的去修改day的值, C++则会报错,说明我们的改动不合规矩



示例2:

```
1 #include <iostream>
2 using namespace std;
3
4 #define day 7
5 int main()
6 {
7     const int year = 365;    //这里修饰了year, 后面无法修改
8     cout<<"一周有"<<day<<"天"<<endl;
9     cout<<"一年有"<<year<<"天"<<endl;
10    system("pause");
11    return(0);
12 }
```

1.5 关键字

- 关键字是C++内置的函数或字符名称,我们在创建变量时不用用关键字来给变量来命名

1.6 标识符命名规则

C++在对标识符(变量, 常量)命名时有一套规则, 具体如下:

- 标识符不可以是关键字
- 标识符只能由数字, 字母, 下划线构成

- 第一个字符必须为字母或下划线
- 标识符大小写敏感

建议是标识符名称要通俗易懂，做到见名知意的效果

2 数据类型

C++规定在创建一个标识符的时候必须指定其数据类型，否则无法对该标识符分配内存

2.1 整型

int 作用：整型变量表示的是整数类型的数据

C++共有4种表示整型的方式，他们的区别在于占用空间的不同

数据类型	占用空间	取值范围
short(短整型)	2字节	$(-2^{15} \sim 2^{15}-1)$
int(整型)	4字节	$(-2^{31} \sim 2^{31}-1)$
long(长整型)	windows为4字节，Linux为4字节(32x)或8字节(64x)	$(-2^{31} \sim 2^{31}-1)$
long long(长长整型)	8字节	$(-2^{63} \sim 2^{63}-1)$

2.2 sizeof 关键字

作用：利用sizeof关键字可以统计数据所占的内存大小

语法：sizeof(数据类型/变量)

示例:

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      //可以利用sizeof求出数据类型占用多少内存空间
7      short num1 =10;
8      cout<<"short占用的内存为"<<sizeof(num1)<<endl;
9      int num2 = 10;
10     cout<<"int占用的内存为"<<sizeof(num2)<<endl;
11     long long num3 =10;
12     cout<<"long long 占用的内存为"<<sizeof(num3)<<endl;
13     system("pause");
14     return(0);
15 }
```

- 此时输出的结果如下

```
short占用的内存为2
int占用的内存为4
long long占用的内存为8
```

2.3 实型（浮点型）

作用：用于表示小数

浮点型变量分为两种：

- 单精度float
- 双精度double

两者的区别在于精度和占用内存不同

数据类型	占用大小	精度
float	4字节	7位有效数字
double	8字节	15~16位有效数字

① Note

在使用float时要注意语法 `float num1 = 3.14f`, 只有带上f后才会被认定为float类型，否则会按照默认的double类型赋值

- 表示小数时也可以用科学计数法

```
1 float f3 = 3e2; // 3*10^2
2 cout<<f3<<endl;
3 float f4 = 3e-2; // 3*10^-2
4 cout<<f4<<endl;
```

此时输出的结果便为：

```
300
0.03
```

2.4 字符型

- 作用：字符型变量用于显示单个字符
- 语法：`char ch = 'a'`

tips1: 显示字符型变量的时候只能用单引号，不能用双引号

tips2: 单引号内只能有一个字符，不可以是字符串

- 字符型变量只占用1字节
- 字符型变量将变量以ASCII码的形式储存在内存里

如何查看字符型变量的ASCII码

```
cout<<(int)[变量名]<<endl
```

- 常用ASCII码：a-97 A-65

2.5 转义字符

作用：表示一些不能显示出来的ASCII字符

常用的转义字符有：\n \\ \t

转义字符	作用	ASCII码
\n	换行符	010
\\	转义一个\	092
\t	水平制表符(占8个位置)	009

2.6 字符串型

- 作用：用于表示一串字符

两种风格

1. C语言风格：char 变量名[] = "字符串值"

2. C++风格：string 变量名 = "字符串值"

```
1  int main()
2  {
3      char str/*字符串名*/[] = "hello world"; //tips1:字符串名后要加[]
4      cout << str <<endl;
5
6      string str2 = "114514";    //要包含一个头文件#include <string>
7      cout<<str2<<endl;
8      system("pause");
9
10     return(0);
11 }
```

Important

C++风格字符串需要在开头加入头文件#include<string>

2.7 布尔类型(bool)

作用：作用于条件判断，代表真或假

- bool类型只有两个值
- True——真(1)
- False——假(0)
- bool 占用1字节的内存空间

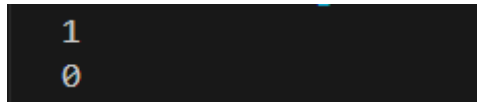
示例：

```

1  int main()
2  {
3      bool flag = true ; //true代表真，本质上是"1"
4      cout<<flag<<endl;
5      bool flag2 = false ;//false代表假，本质上是"0"
6      cout << flag2<<endl;
7      system("pause");
8
9      return(0);
10 }

```

而这块代码的输出结果



```

1
0

```

2.8 数据的输入

作用：从键盘上获取数据

语法：cin >> 变量

示例：

```

1  int main()
2  {
3      //整型
4      int a = 0;
5      cout << "请键入整型变量a的值"<<endl;
6      cin >> a;
7      cout <<a<<endl;
8      //浮点型
9      float f =1.14f;
10     cout<<"请给浮点型f赋值"<<endl;
11     cin >> f;
12     cout <<f<<endl;
13     //字符串型
14     string str = "hello world";
15     cout<<"输入你的字符串值"<<endl;
16     cin >> str;
17     cout<<str<<endl;
18
19
20     system("pause");
21
22     return(0);
23 }

```

输出结果为:


```
请键入整型变量a的值
12
12
请给浮点型f赋值
43.2
43.2
输入你的字符串值
nihao
nihao
```

3 运算符

作用：用于执行代码的计算

主要有一下几种运算符：

运算符类型	作用
算术运算符	用于处理四则运算
赋值运算符	用于将表达式的值赋给变量
比较运算符	用于表达式的比较，返回一个真值或假值
逻辑运算符	用于根据表达式的值返回真值或假值

3.1 算术运算符

作用：用于处理四则运算

包括一下符号：

运算符	术语	示例	结果
+	正数	+3	+3
-	负数	-4	-4
+	加号	4+5	9
-	减号	6-3	3
*	乘号	6*7	42
/[^1]	除号	94/7	13
%	取模(取余)	10%3	1
++	前置递增	a=2 b=++a	a=3 b=3
++	后置递增	a=2 b=a++	a=3 b=2
--	前置递减	a=2 b--a	a=1 b=1
--	后置递减	a=2 b=a--	a=1 b=2

示例1 四则运算的示例

```
1  int main()
2  {
3      int a1 = 10;
4      int b1 = 7;
5      cout<<a1 + b1 <<endl;
6      cout<<a1 - b1 <<endl;
7      cout<<a1 * b1 <<endl;
8      cout<<a1 / b1 <<endl; //这里为整除运算，结果也会为整数
9
10     float a2 ;
11     float b2 ;
12     cout<<"请输入两个浮点数"<<endl;
13     cin >> a2;
14     cin >> b2;
15     cout<< "a2除以b2的值为"<<a2 / b2<<endl; //这里是非整除
16
17     system("pause");
18     return(0);
19 }
```

- 取模运算本质就是取余数
- 两个小数之间不能做取模运算

前置递增与后置递增

- 前置，后置递增都是使变量进行加一的操作
- 前置递增: 先对变量进行递增，再进行表达式运算
- 后置递增: 先进行表达式的运算，再对变量递增

```
1  int main()
2  {
3      //前置运算
4      int a = 10;
5      int b = 3;
6      int r1 = ++a * b;
7      cout<<"r1="<< r1 <<endl;
8
9      //后置运算
10     int a2 = 10;
11     int b2 = 3;
12     int r2 = a2++ * b2;
13     cout<<"r2="<< r2 <<endl;
14     cout<<"a2="<<a2<<endl;
15
16     system("pause");
17     return(0);
18 }
```

```
r1=33
r2=30
a2=11
```

- 我们不难发现，在上述代码运算过程中我们的"a"变量先被加1再参与到了运算之中，而我们的"a2"变量则是再运算结束后才被加1

3.2赋值运算符

作用：将表达式的值赋给变量

主要包括以下几个符号：

运算符	术语	示例	结果
=	赋值	a=10	a=10
+=	加等于	a=10 a+=2	a=12
-=	减等于	a=10 a-=2	a=8
=	乘等于	a=10 a=2	a=20
/=	除等于	a=10 a/=2	a=5
%=	取模等于	a=10 a%=2	a=0

3.3 比较运算符

作用：用于比较表达式的真假，并返回一个真值或假值

主要有以下的符号：

运算符	术语	示例	结果
==	相等于	4 == 3	0
!=	不等于	4 != 3	1
<	小于	4 < 3	0
>	大于	4 > 3	1
<=	小于等于	4 <= 3	0
>=	大于等于	4 >= 3	1

*tips：*再代码中由于有优先级的影响，我们可以这么提升运算优先级 `cout << (a == b)<< endl;`

3.4 逻辑运算符

作用：用于根据表达式的值返回真值或假值

主要有以下符号：

运算符	术语	示例	结果
!	非	!a	如果a为假，则!a为真；如果a为真，则!a为假

运算符	术语	示例	结果
&&	与	a&&b	如果a和b都为真，则结果为真，否则为假
	或	a b	如果a和b中有一个为真，则结果为真，二者都为假的时候，结果为假

3.4.1 逻辑非

```

1  int main()
2  {
3      //逻辑非
4      int a= 10 ;
5      cout << !a << endl;
6      //运算结果为 0（解释：在C++中，只要结果不为0，都视为真，故输出结果为假）
7      cout << !!a << endl;
8      //结果为1，取了两次反（从真变假再变真）
9      system("pause");
10     return(0);
11 }
```

总结:真变假，假变真

3.4.2 逻辑与

```

1  int main()
2  {
3      // 逻辑与
4      int a = 10;
5      int b =10;
6      cout << (a&&b) << endl; //此处也要优先运算
7      //运算结果为 1（真）
8      a = 10;
9      b = 0;
10     cout << (a&&b) << endl;
11     //运算结果为 0（假）
12     a = 0;
13     b = 0;
14     cout << (a&&b)<<endl;
15     //运算结果为 0（假）
16     system("pause");
17     return(0);
18 }
```

总结: 同真为真，其余为假

3.4.3 逻辑或

```

1  int main()
2  {
3      //逻辑或
4      int a = 10;
5      int b = 10;
```

```

6      cout << (a || b) << endl;
7      //结果为1
8
9      a = 0;
10     b = 10;
11     cout << (a || b) << endl;
12     //结果仍为1
13     a=0;
14     b=0;
15     cout << (a || b) << endl;
16     //结果为0
17     system("pause");
18     return(0);
19 }

```

总结: 同假为假, 其余为真

4 程序流程结构

C/C++支持的三种程序运行结构: **顺序结构**, **选择结构**, **循环结构**

- **顺序结构**: 类似于Python的运行结构, 程序按顺序执行, 不发生跳转
- **选择结构**: 依据条件是否满足, 有选择的执行相应功能
- **循环结构**: 依据条件是否满足, 循环多次执行某段代码

4.1 选择结构

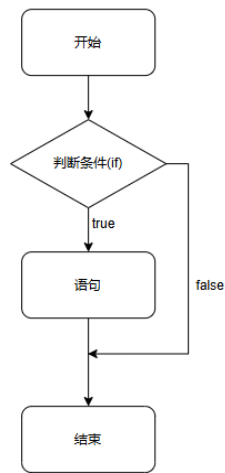
4.1.1 if语句

作用: 执行满足条件的语句

其主要有三种形式:

- 单行格式if语句
- 多行格式if语句
- 多条件的if语句

1. 单行格式if语句: `if(条件){ 条件满足执行的语句 }`



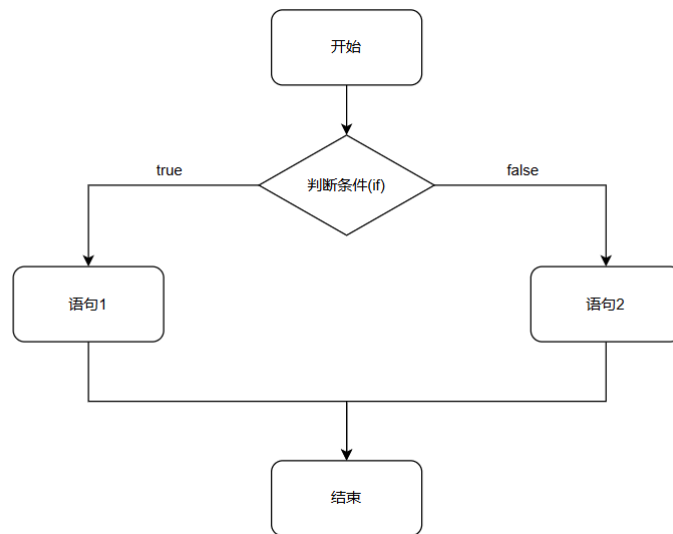
示例:

```
1  int main()
2  {
3      //单行if结构
4      //让用户输入一个分数，如果大于600，则输出"恭喜"
5      //1. 用户输入分数
6      int score = 0;
7      cout << "请输入一个分数"<<endl;
8      cin >> score ;
9      //2. 打印用户分数
10     cout << "您的分数为:"<<score<<endl;
11     //3. 判断
12     if(score > 600) //if条件语法后没有分号!!!
13     {
14         cout<<"恭喜"<<endl;
15     }
16
17     system("pause");
18     return(0);
19 }
```

Important

if条件后面不要加分号

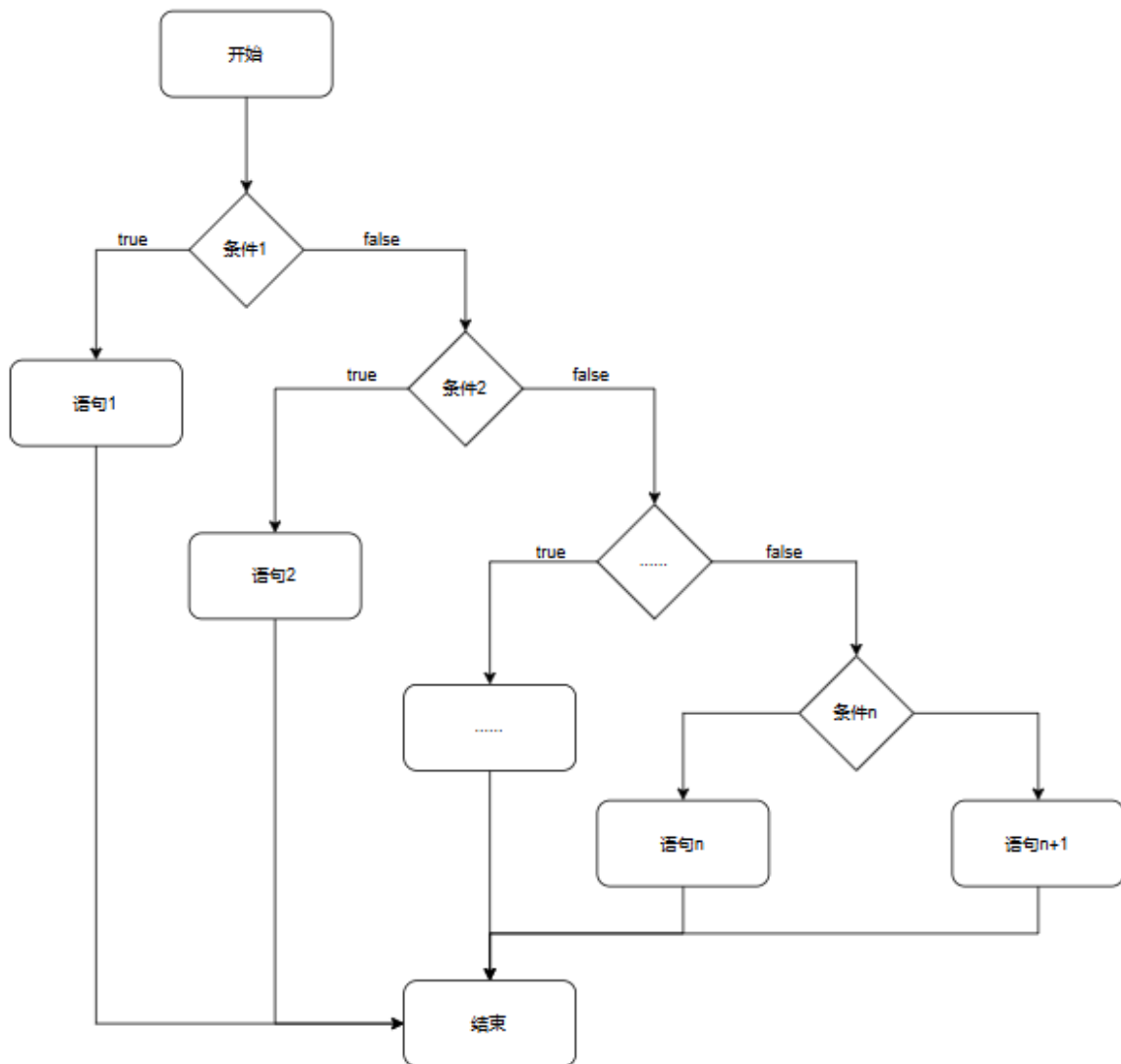
2. 多行格式if语句: `if(条件){ 条件为真执行的语句 }else{ 条件不满足执行的语句 }`



示例:

```
1  int main()
2  {
3      //多行格式if
4      //提示用户输入分数，如果分数大于600，则输出"恭喜"，若没有大于600，则输出"别放弃"
5      int score = 0;
6      cout << "请输入一个分数"<<endl;
7      cin >> score;
8      cout << "您的分数为"<< score <<endl;
9      //执行判断
10     if(score > 600) //大于600的情况
11     {
12         cout << "恭喜"<<endl;
13     }
14     else //小于600的情况
15     {
16         cout << "别放弃"<<endl;
17     }
18     system("pause");
19     return(0);
20 }
```

3. 多条件的if语句: `if(条件1){满足条件1执行的语句}else if(条件2){满足条件2执行的语句}... else{都不满足执行的语句}`



```

1  int main()
2  {
3      //多条件的if语句
4      //1. 输入分数
5      int score = 0;
6      cout << "请输入一个分数"<<endl;
7      cin >> score;
8      cout << "您的分数是" << score << endl;
9      //2. 条件判断(大于600)
10     if (score >= 600)
11     {
12         cout << "恭喜"<<endl;
13     }
14     else if (score >= 500)//这里不能写(500 < score < 600)
15     {
16         cout << "别放弃"<<endl;
17     }
18     else if (score >= 400)
19     {
20         cout << "还可以"<<endl;
21     }
  
```



```

22     else
23     {
24         cout << "别摆烂辣! "<<endl;
25     }
26
27     system("pause");
28     return(0);
29 }

```

嵌套if语句：在if语句中再嵌套一个if语句

案例要求:

- 在上个代码的基础上，根据分数再细化
- 大于700为特等，大于650为一等，大于600为优秀

```

1  int main()
2  {
3      //多条件的if语句
4      //1. 输入分数
5      int score = 0;
6      cout << "请输入一个分数"<<endl;
7      cin >> score;
8      cout << "您的分数是" << score << endl;
9      if (score >= 600)
10     {
11         cout << "恭喜"<<endl;
12         if (score >= 700) //嵌套的if语句
13         {
14             cout << "特等"<<endl;
15         }
16         else if (score >= 650)
17         {
18             cout << "一等"<<endl;
19         }
20         else
21         {
22             cout << "优秀"<<endl;
23         }
24     }
25     else if (score >= 500)//这里不能写(500 < score < 600)
26     {
27         cout << "别放弃"<<endl;
28     }
29     else if (score >= 400)
30     {
31         cout << "还可以"<<endl;
32     }
33     else
34     {
35         cout << "别摆烂辣! "<<endl;
36     }
37
38     system("pause");
39     return(0);

```

练习见[[练习.md]]

4.1.2 三目运算符

作用:通过三目运算符实现简单的判断

语法: 表达式1 ? 表达式2 : 表达式3

解释:

如果表达式1的值为真, 执行表达式2, 并返回表达式2的结果;

如果表达式1的值为假, 执行表达式3, 并返回表达式3的结果;

示例:

```
1  int main()
2  {
3      //三目运算符
4      int a =30;
5      int b =20;
6      int c =0;
7      c=(a > b ? a : b);
8      cout << c <<endl;
9
10     //在C++中, 三目运算符返回的是变量, 可以继续赋值
11     (a > b ? a:b)=100; //a和b做大小比较, 较大的变量被赋值为100
12     system("pause");
13
14     return(0);
15 }
```

4.1.3 switch语句

执行多条件分支语句

语法:

```
1  switch(表达式)
2
3  {
4      case 结果1 : 执行语句;break;
5
6      case 结果2 : 执行语句;break;
7
8      ...
9
10     default : 执行语句;break;
11 }
```

示例:

```
1  int main()
2  {
```

```

3      //给电影评分
4      //9-10 经典
5      //7-8 非常好
6      //5-6 不错
7      // <5 不好
8
9      cout << "请给电影打分"<<endl;
10     int score = 0;
11     cin >> score ;
12     cout << "您的打分为"<<score<<endl;
13     switch (score)
14     {
15     case 10 :
16         cout << "经典"<<endl;
17         break; //退出当前分支
18     case 9 :
19         cout << "经典"<<endl;
20         break;
21     case 8 :
22         cout << "非常好"<<endl;
23         break;
24     case 7 :
25         cout << "非常好"<<endl;
26         break;
27     case 6 :
28         cout << "一般"<<endl;
29         break;
30     case 5 :
31         cout << "一般"<<endl;
32         break;
33     default:
34         cout << "不好"<<endl;
35         break;
36     }
37     system("pause");
38
39     return(0);
40 }

```

记得要写break;

缺点:switch判断的时候只能是整型或字符型，不可以是一个区间

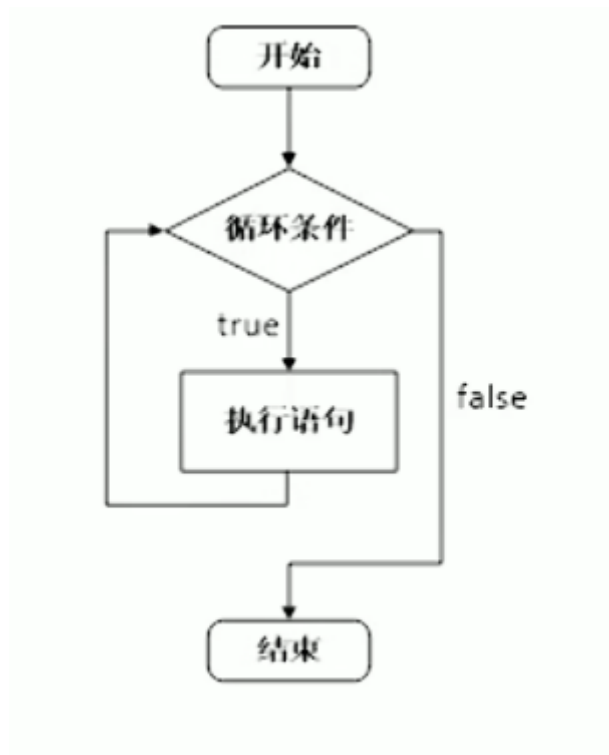
优点:结构清晰，执行效率高(速度快)

4.2 循环结构

4.2.1 while循环语句

作用:满足循环条件，执行循环语句

语法: while(循环条件){循环语句}



解释:只要循环条件为真,就执行循环语句

示例:

```
1  int main()
2  {
3      //在屏幕中打印0-9这10个数字
4
5      int num = 0;
6      while (num < 10)
7      {
8          cout << num <<endl;
9          num++;
10     }
11     system("pause");
12
13     return(0);
14 }
```

如果while 后条件为(1), 则为无限循环, 要避免死循环的出现

练习:猜数游戏

```
1  int main()
2  {
3      //添加随机数种子
4      srand((unsigned int)time(NULL));
5      int num2 = rand()%100 + 1 ;
6      //cout << num2 <<endl;
7      int val = 0;
8      cout << "请输入一个数开始猜数游戏"<<endl;
9      while (val != num2)
10     {
11         cin >> val ;
12         if (val > num2 )
```

```

13     {
14         cout << "猜大辣，再来一次吧"<<endl;
15     }
16     else if (val < num2)
17     {
18         cout << "猜小辣，再来一次吧"<<endl;
19     }
20
21 }
22 cout << "厉害，对辣"<<endl;
23
24 system("pause");
25 return(0);
26 }

```

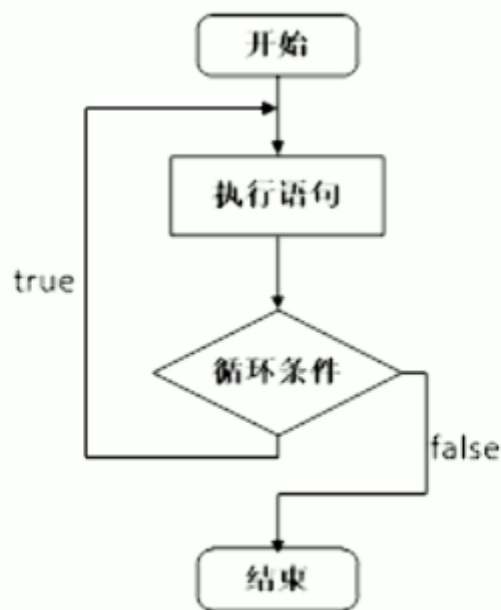
4.2.2 do...while循环

作用：满足循环条件，执行循环语句

语法：do{循环语句}while(循环条件);

④ Note

与while不同的是，do...while会先执行一次循环语句，再判断循环条件



示例：

```

1  int main()
2  {
3      //do while 循环
4      int num =0;
5      do
6      {
7          cout << num <<endl;
8          num++;
9      } while (num < 10);
10     system("pause");
11     return(0);
12
13 }

```

练习案例：水仙花数

- 水仙花数是一个三位数，它的每个位上的三次幂之和等于它本身
- 例： $1^3+5^3+3^3=153$

```

1  //注意：要有#include<cmath>
2  int main()
3  {
4      //定义初始值，其中fnum3,2,1分别代表百十个位，这个三位数要和初始值fnum相等
5      int fnum1 = 0;
6      int fnum2 = 0;
7      int fnum3 = 1;
8      float fnumt = 0;
9      int fnum = 100;
10     do
11     {
12
13         fnum++;
14         fnum1++;
15         //三位数输出，个十百位分别输出
16         if (fnum1 - 1 == 9)
17         {
18             fnum1 = 0;
19             fnum2++;
20         }
21         if (fnum2 - 1 == 9)
22         {
23             fnum2 = 0;
24             fnum3++;
25         }
26         if (fnum3 - 1 == 9)
27         {
28             fnum3 = 0;
29         }
30
31         fnumt = pow(fnum1,3) + pow(fnum2,3) + pow(fnum3,3);
32         if ( fnumt == fnum)
33         {
34             cout <<"水仙花"<< fnum << endl;
35         }

```

```

36         // cout << fnum << endl;
37         // cout << fnumt << endl;
38         // cout << "个位" << fnum1 << endl;
39         // cout << "十位" << fnum2 << endl;
40         // cout << "百位" << fnum3 << endl;
41
42     } while (fnum < 1000);
43     system("pause");
44     return(0);
45
46 }

```

Important

如何获取一个三位数的个十百位？

- 例:153
- 个位: $153 \% 10 = 3$ 对数字取模于10可以获得个位
- 十位: $153 / 10 = 15 \rightarrow 15 \% 10 = 5$ C++中整除只留整数部分即 $(153 / 10) \% 10$
- 百位: $153 / 100 = 1$

示例:

```

1  int main()
2  {
3      int num = 100;
4      do{
5          num++;
6          if(num == pow(num%10, 3) + pow((num/10)%10, 3) + pow(num/100, 3))
7          {
8              cout << num << endl;
9          }
10     }while(num < 999);
11
12     system("pause");
13     return(0);
14 }

```

4.2.3 for 循环

作用: 满足循环条件, 执行循环语句

语法: `for(起始表达式; 条件表达式; 末尾循环体){循环语句;}`

- 起始表达式不参加循环
- 条件表达式确定循环条件
- 一次循环执行后执行末尾循环体

示例:

```

1  int main()
2  {
3      for (int i = 0; i < 10; i++)
4      {
5          cout << i <<endl;
6      }
7
8      system("pause");
9      return(0);
10 }

```

① Note

对for(a;b;c){d}来看，执行顺序如下

1. 先执行一次 a
2. 判断 b
3. 若b为真，重复2, 3, 4, 5；否则退出循环
4. 执行 d
5. 执行 c

for 循环结构简单，比较常用

练习案例：敲桌子

- 输出1~100，若该数个位含有7，或10位含有7，或该数字是7的倍数，则我们输出敲桌子，其余数字直接打印

```

1  int main()
2  {
3      for (int i = 0; i < 100; i++)
4      {
5          if (i%10 == 7)
6          {
7              cout << "敲桌子" <<endl;
8          }
9          else if ((i/10)%10 == 7)
10         {
11             cout << "敲桌子" <<endl;
12         }
13         else if ( i%7 == 0 )
14         {
15             cout << "敲桌子" <<endl;
16         }
17         else
18         {
19             cout << i <<endl;
20         }
21     }
22 }

```

① Note

if比较语句中我们可以用逻辑运算符来提高if语句的精确性

比如上面的示例中多个if便可以写成 `if(i % 10 == 7 || i%7==0 || (i/10)%10==7)`

4.2.4 嵌套循环

- 在循环体中再次嵌套循环，用于解决实际问题

示例

```
1  int main()
2  {
3      for (int i = 0; i < 10 ; i++) //外层循环
4      {
5          for (int j = 0; j < 10; j++) //内层循环
6          {
7              cout << "*";
8          }
9          cout << endl;
10     }
11     // 外层走一次，内层走一周
12
13     system("pause");
14     return(0);
15 }
```

案例：乘法口诀表

- 打印九九乘法表

```
1  int main()
2  {
3      //九九乘法表，实际上就是 行 x 列 = 数字，即将行和列表示出来即可
4      for (int i = 0; i < 10; i++) // i 代表 行
5      {
6          for (int j = 1; j < i+1 ; j++) // j 代表 列
7          {
8              cout << j << "x" << i << "=" << i*j << " ";
9          }
10         cout << endl;
11     }
12     system("pause");
13     return(0);
14 }
```

输出呈现：

```
1X1=1
1X2=2 2X2=4
1X3=3 2X3=6 3X3=9
1X4=4 2X4=8 3X4=12 4X4=16
1X5=5 2X5=10 3X5=15 4X5=20 5X5=25
1X6=6 2X6=12 3X6=18 4X6=24 5X6=30 6X6=36
1X7=7 2X7=14 3X7=21 4X7=28 5X7=35 6X7=42 7X7=49
1X8=8 2X8=16 3X8=24 4X8=32 5X8=40 6X8=48 7X8=56 8X8=64
1X9=9 2X9=18 3X9=27 4X9=36 5X9=45 6X9=54 7X9=63 8X9=72 9X9=81
Press any key to continue . . .
```

4.3 跳转语句

4.3.1 break 语句

作用：跳出选择结构或循环结构

break使用的时机：

- 出现在switch语句中，终止case并跳出switch
- 出现在循环语句中，作用是跳出当前循环语句
- 出现在嵌套循环中，作用是跳出最近的内层循环语句

示例1：

```
1  int main()
2  {
3      for (int i = 0; i < 10 ; i++)
4      {
5          cout << i << endl;
6          if (i == 5)
7          {
8              break;
9          }
10
11     }
12     system("pause");
13     return(0);
14 }
```

示例2

```
1  int main()
2  {
3      for (int i = 0; i < 10 ; i++)
4      {
5          for (int j = 0; j < 10; j++)
6          {
7              if (j == 5)
```

```

8         {
9             break;
10        }
11        cout << "*";
12    }
13    cout<< endl;
14 }
15 system("pause");
16 return(0);
17 }

```

4.3.2 continue 语句

作用:在循环语句中，跳过本次循环中余下的未执行的代码，继续执行下一次循环

示例:

```

1  int main()
2  {
3      for (int i = 0; i <= 100; i++)
4      {
5          if (i%2 == 0)
6          {
7              continue;
8          }
9
10         cout << i << endl;
11     }
12     system("pause");
13     return(0);
14 }

```

- 实现了0~100奇数的输出

4.3.3 goto 语句

作用: 可以无条件跳转语句

语法: goto 标记

- 标记一般用纯大写英文表示
- goto 语法尽量不要经常使用，以免造成代码逻辑混乱
- 标记定义 T:

5 数组

5.1 概述

数组就是一个集合，里面存放了相同类型的数据元素

- **特点1:** 数组中每个数据元素都是相同的数据类型
- **特点2:** 数组是连续的内存位置组成的

5.2 一维数组

5.2.1 一维数组的定义方式:

一维数组有三种定义方式:

1. 数据类型 数组名[数组长度]
2. 数据类型 数组名[数组长度]={ 值1,值2,...}
3. 数据类型 数组名[]={ 值1,值2,...}

示例1:

```
1  int main()
2  {
3      // 1. `数据类型 数组名[ 数组长度 ]`
4      int arr[5];
5      arr[0] = 10;
6      arr[1] = 20;
7      arr[2] = 30;
8      arr[3] = 40;
9      arr[4] = 50;
10     int a = 3;
11     cout << arr[3] << endl; //这个访问的是 40
12     cout << arr[ a ] << endl; //数组的下标可以通过变量来索引
13     system("pause");
14     return(0);
15 }
```

示例2:

```
1  int main()
2  {
3      //2. 数据类型 数组名[ 数组长度 ]={ 值1,值2,...}
4      int arr2[5] = {10,20,30,40,50};
5      for (int i = 0; i < 5; i++) // i < 5 中, 5表示数组长度
6      {
7          cout << arr2[i] << endl;
8      }
9      //若初始没有补齐数据, 会用0来填充空余数据
10     system("pause");
11     return(0);
12 }
```

示例3:

```

1  int main()
2  {
3      //3.数据类型 数组名[]={ 值1,值2,...}
4      //定义数组时必须给初始长度
5      int arr3[] = {1,1,4,5,1,4};
6      for (int i = 0; i < 6; i++)
7      {
8          cout << arr3[i]<<endl;
9      }
10     system("pause");
11     return(0);
12 }

```

- 数组中的数据是从0开始标记(索引)下标
- 我们可以通过下标来访问数组中的元素

5.2.2 一维数组数组名

用途:

1. 可以统计整个数组在内存中所占的长度
2. 可以获取数组在内存中的首地址

对1: `sizeof(数组名)`

我们可以用 `sizeof(arr)/sizeof(arr[0])` 来获取内存的长度

对2: `cout << arr <<endl;`

一般该地址为16进制地址

```

1  int main()
2  {
3      //1. 查询数组所占内存大小
4      int arr[6]={1,1,4,5,1,4};
5      cout << "数组大小为"<< sizeof(arr)<<endl;
6      cout << "数组长度为"<< sizeof(arr)/sizeof(arr[0])<<endl;
7      //2. 查看首地址
8      cout << "内存地址"<< arr << endl;//16进制
9      cout << (long long)arr <<endl;//强转10进制
10     cout << &arr[0]<<endl;//数组中某个元素的内存地址
11     cout << (long long)&arr[0] <<endl;//数组中某个元素的10进制内存地址
12     //使用 long long 包容16进制精度问题
13     cout << &arr[1]<<endl;//第二个元素位置
14     cout << (long long)&arr[1] <<endl;//10进制
15     //第二个与第一个相差4字节
16     //数组名是常量, 不能修改赋值
17     system("pause");
18     return(0);
19 }

```

练习案例1:

- 在一个数组中记录了5个数据, `arr[5] = {10,30,20,70,60}`
- 找出这个数组中最大数

```

1  int main()
2  {
3      int arr[5]= {10,30,60,40,20};
4      int max = 0; //假设某一最大值
5      for (int i = 0; i < 5; i++)//访问数组中的每一个数
6      {
7          if (arr[i] > arr[max]) //比较假设值和访问值大小
8          {
9              max = i ; //若大于，则替换假设最大值
10         }
11     }
12     //循环结束时，最大值以确定
13     cout << "最大的数是" << arr[max]<<endl; //输出
14
15     system("pause");
16     return(0);
17 }

```

Note

在上面代码中，for循环内部也可以使用三目运算来找最大值

```
max = (arr[max] > arr[i] ? max : i ); //使用三目运算符
```

练习案例2：数组元素逆置

- 声明一个5个元素的数组，并将其逆置
- 示例：原数组 {1,3,4,2,3} ==> 输出{3,2,4,3,1}
- 输出逆置

```

1  int main()
2  {
3      int arr[5] = {1,3,4,2,3};
4      for (int i = 0; i < 5; i++)
5      {
6          cout << arr[4-i] ;
7      }
8      cout << endl;
9
10     system("pause");
11     return(0);
12 }

```

- 创立逆置数组

```

1  int main()
2  {
3      int arr[5] = {1,3,4,2,3};
4      int arrt[5];
5      int t; //建立逆置变量
6      for (int i = 0; i < 5; i++)

```

```

7      {
8          t = (sizeof(arr)/sizeof(arr[0]))-1-i; //实现逆置变量
9          arrt[i] = arr[t]; //实现原数组向逆置数组的赋值
10     }
11     //逆置数组建立完成，以下为检查
12     for (int i2 = 0; i2 < 5; i2++)
13     {
14         cout << arrt[i2] <<endl;
15     }
16
17     system("pause");
18     return(0);
19 }

```

- 原数组的逆置

```

1  int main()
2  {
3      int arr[5] = {1,3,4,2,3};
4      int sta = 0;
5      int end = sizeof(arr)/sizeof(arr[0])-1;
6      int temp = 0;
7      //核心
8      for ( ; sta < end ; ) //当起始值位置大于末尾值位置时停止
9      {
10         temp = arr[sta]; //初始值赋值至临时内存
11         arr[sta] = arr[end]; //末尾值赋值至初始值
12         arr[end] = temp; //初始值(临时)赋值至末尾值
13         sta++; //初始值后移一位
14         end--; //末尾值前移一位
15     }
16     //数组倒置结束
17     for (int i2 = 0; i2 < 5; i2++)
18     {
19         cout << arr[i2]<<endl;
20     }
21
22     system("pause");
23     return(0);
24 }

```

5.2.3 冒泡排序

作用：最常用的排序算法，对数组内的元素进行排序

1. 比较相邻的元素，如果第一个比第二个大，就交换他们
2. 对每一对相邻元素做同样工作，执行完毕后，找到第一个最大值
3. 重复以上步骤，每次比较次数-1，直到不需要比较

示例：将数组{4,2,3,0,5,7,1,3,9}升序排列

```

1  int main()

```

```

2  {
3      int arr[9] = {4,2,3,0,5,7,1,3,9};
4      for (int i = 0; i < (sizeof(arr)/sizeof(arr[0])-1); i++)//排序的总轮数=元素
        个数-1
5          {
6              for (int j = 0; j <(sizeof(arr)/sizeof(arr[0])-1)-i ; j++) //每轮排序
                的次数 = 元素个数 -1 -当前轮数
7                  {
8                      if (arr[j] > arr[j+1] ) //判断相邻的两个数的大小
9                          {
10                             //实现交换
11                             int temp = arr[j];
12                             arr[j] = arr[j+1];
13                             arr[j+1] = temp;
14                         }
15                 }
16         }
17     //输出验证
18     for (int i2 = 0; i2 < 9; i2++)
19     {
20         cout << arr[i2]<<endl;
21     }
22
23     system("pause");
24     return(0);
25 }

```

- 利用遍历实现数据筛查
- 题目来源[洛谷P1085 \[NOIP2004 普及组\] 不高兴的津津](#)

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int a,b,t;
7      int m =0;
8      int arr[7];
9      for (int i = 0; i < 7; i++)
10         {
11             cin >> a >> b;
12             int k =a +b;
13             arr[i] = k; //将获得的数据记入数组
14         }
15     //对数组遍历，找出最大的那个数
16     for (int j = 0; j < 7; j++) //假设一个最大值 arr[0]，让arr[0]和下一个数比较，
        若大于，则将m赋值为j
17     {
18         if (arr[m] < arr[j])
19         {
20             m = j;
21             t = arr[j];
22         }
23     }
24     if ( t > 8)

```



```

25     {
26         cout << m+1 <<endl;
27     }
28     else
29     {
30         cout << 0 <<endl;
31     }
32
33     system("pause");
34     return(0);
35 }

```

5.3 二维数组

5.3.1 二维数组的定义方式：

- 数据类型 + 数组名[行数][列数];
- 数据类型 + 数组名[行数][列数] = {数据1, 数据2}, {数据3, 数据4}; [^2]
- 数据类型 + 数组名[行数][列数] = {数据1, 数据2, 数据3, 数据4};
- 数据类型 + 数组名[][列数] = {数据1, 数据2, 数据3, 数据4};
- 第三和第四组会自动区分行列数(即从**第一个数据**开始计数，记到列数自动换行)

5.3.2 二维数组的赋值方式

- arr[0][0] = 元素;
- arr[0][1] = 元素;
-

如何输出一个二维数组？

写一个嵌套循环，外层打印行数，内层打印列数

```

1  for (int i = 0; i < count ; i++)
2  {
3      for(int j = 0; j < count ; j++)
4      {
5          cout << arr[i][j];
6      }
7      cout << endl;
8  }

```

① Note

- 直观表示一个二维数组：`int arr[3][3];`

列\行	0列	1列	2列
0行	arr[0][0]	arr[0][1]	arr[0][2]
1行	arr[1][0]	arr[1][1]	arr[1][2]

列\行	0列	1列	2列
2行	arr[2][0]	arr[2][1]	arr[2][2]

行列式行列式，先行后列

5.3.3 二维数组数组名

- 查看二维数组所占内存空间
- 获取二维数组首地址
- 具体如下

```
1  int main()
2  {
3      int arr[3][3] =
4      {
5          {1,1,4},
6          {5,1,4}
7      };
8      //1. 查看占用内存空间大小
9      cout << sizeof(arr) << endl; // out : 36 (6*6)
10     cout << sizeof(arr[0][0]) << " " << sizeof(arr[0]) << endl; //out : 4 12(单个
        元素 第一行)
11     //我们可以通过以上数据获得行数与列数
12     sizeof(arr)/sizeof(arr[0]); //列数
13     sizeof(arr[0])/sizeof(arr[0][0]); //列数
14
15     //2. 查看首地址
16     cout << (long long)arr << endl; //out : 6422000
17     cout << (long long)arr[0] << endl; //二维数组地址与arr[0][0]首地址重合
18     cout << (long long)arr[1] << endl; //out : 6422012 差12(3*4)
19     cout << (long long)&arr[0][0] << endl; //二维数组地址与arr[0][0]首地址重合
20     cout << (long long)&arr[0][1] << endl; //out : 6422004 与[0][0]差4
21
22     system("pause");
23     return(0);
24 }
```

5.3.3 二维数组应用案例

考试成绩统计

- 有三名同学(A,B,C)，在一次考试中成绩分别如下，**请输出三名同学的总成绩**

	语文	数学	英语
A	100	100	100
B	90	50	100
C	60	70	80

```
1  int main()
2  {
```

```

3
4     int arr[3][3] =
5     {
6         {100,100,100},
7         {90,50,100},
8         {60,70,80}
9     };
10
11     for (int i = 0; i < 3; i++)
12     {
13         int temp = 0;
14         for (int j = 0; j < 3; j++)
15         {
16             temp += arr[i][j];
17         }
18         cout << temp <<endl;
19     }
20
21     system("pause");
22     return(0);
23 }

```

5.3.4 二维数组排序

- 核心思路：冒泡排序

```

1  int main()
2  {
3      int l,m;
4      cin >> l >> m;
5      int arr[m][2];
6      for (size_t i = 0; i < m; i++)
7      {
8          for (size_t j= 0; j < 2; j++)
9          {
10             cin >> arr[i][j];
11         }
12     }
13
14     for (size_t i2 = 0; i2 < m-1; i2++)
15     {
16         for (size_t j = 0; j < m-i2-1; j++)
17         {
18             if (arr[j][0] > arr[j+1][0])
19             {
20                 int temp1 = arr[j][0];
21                 int temp2 = arr[j][1];
22                 arr[j][0] = arr[j+1][0];
23                 arr[j][1] = arr[j+1][1];
24                 arr[j+1][0] = temp1;
25                 arr[j+1][1] = temp2;
26             }
27         }
28     }
29 }

```

```

30     for (size_t i = 0; i < m; i++)
31     {
32         for (size_t ij = 0; ij < 2; ij++)
33         {
34             cout << arr[i][ij]<<" ";
35         }
36         cout <<endl;
37     }
38
39
40     return 0;
41 }

```

6 函数

6.1 概述

作用： 将经常使用的一段代码封装起来，减少重复代码

一个较大的程序，一般分为若干个程序块，每个模块实现特定的功能

6.2 函数的定义

一般函数定义有5个主要步骤

1. 返回值类型
2. 函数名
3. 参数列表
4. 函数体语句
5. return表达式

语法

```

1  //返回值类型 函数名(参数列表)
2  int isprime(int n)
3  {
4      //函数体语句
5
6      //return表达式
7      return 0;
8  }
9

```

- 实例1

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int add(int a , int b)

```

```

5  {
6      int sum = a+b;
7      return sum;
8  }
9
10 int main()
11 {
12     int num1,num2;
13     scanf("%d %d",&num1,&num2);
14     printf("%d",add(num1,num2));
15
16     system("pause");
17     return(0);
18 }

```

a,b 我们可以称为形参，num1,num2 我们可以称为实参，函数调用本质是将实参传递给形参并进行函数运算，返回return值

6.3 函数的调用

功能：使用定义好的函数

语法：函数名(参数)

6.4 值传递

- 值传递就是函数调用时实参将数值传递给形参
- 值传递时，形参发生变化，并不会影响实参

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  void swap(int a,int b) //在无需返回值时，可以输入void类型
5  {
6      int temp = a;
7      a = b;
8      b =temp;
9      cout << "交换后:"<<a <<" "<<b <<endl;
10     return;
11 }
12
13 int main()
14 {
15     int i1 = 4;
16     int i2 = 5;
17     cout <<"交换前:"<<i1<<" "<<i2<<endl;
18     swap(4,5);
19 }

```

① Note

在值传递的时候，为实参和形参分别分配内存空间，将实参的内存传递给形参，进而使用形参的内存去执行函数，实参的内存不会发生改变

6.5 函数的常见样式

1. 无参无返
2. 有参无返
3. 无参有返
4. 有参有返

- 实例

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  //1.无参无返
5  void test_01()
6  {
7      cout << "跟你爆了"<<endl;
8      return;
9  }
10
11 //2.有参无返
12 void test_02(int a)
13 {
14     cout << a*a <<endl;
15     return;
16 }
17
18 //3.无参有返
19 int test_03()
20 {
21     return 1000;
22 }
23
24 //4.有参有返
25 int test_04(int k)
26 {
27     return (k*2)+k;
28 }
29
30 int main()
31 {
32     test_01();
33     test_02(4);
34     int num1 = test_03();
35     cout<<num1<<endl;
36     int m = test_04(4);
37     cout <<m <<endl;
38     system("pause");
39     return(0);
40 }
```

6.6 函数的声明

作用：告诉编译器函数名称及如何调用函数，函数的实际主体可以单独定义

- 函数可以声明多次，但函数的定义只能有一次

示例：

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  //声明
5  int max01(int a,int b);
6
7  int main()
8  {
9      int t = max01(5,6);
10     cout << t << endl;
11     system("pause");
12     return(0);
13 }
14
15 //定义
16 int max01(int a,int b)
17 {
18     return a>b ? a : b;
19 }
```

6.7 函数的分文件填写

作用：让代码结构更加清晰

函数分文件编写一般有4个步骤

1. 创建后缀名为.h的头文件
2. 创建后缀名为.cpp的源文件
3. 在头文件中书写函数的声明
4. 在源文件中书写函数的定义

示例

```
1  //head.h
2  #include<bits/stdc++.h>
3  using namespace std;
4
5  void swap(int a,int b);
```

```

1 //fun.cpp
2 #include<bits/stdc++.h>
3 #include "head.h"
4 using namespace std;
5
6 void swap(int a,int b)
7 {
8     int temp = a;
9     a = b;
10    b = temp;
11    cout << a <<" "<<b << endl;
12 }

```

```

1 //test.cpp
2 #include<bits/stdc++.h>
3 using namespace std;
4 #include "head.h"
5
6 int main()
7 {
8     swap(4,5);
9     return(0);
10 }

```

📢 Important

在VSCode中，C++编译只对test.cpp中的main函数进行编译，无法连接到我们的fun.cpp文件

- 解决方法:将头文件的文件目录复制到 .vscode目录下的tasks.json的"args": 的"\${file}"下面即可
- 注意复制的单斜杠要改为多斜杠

7 指针

7.1 指针的基本概念

指针的作用：用于间接访问内存

- 指针的编号是从0开始计数的，一般用16进制表示
- 可以利用指针变量保存地址

7.2 指针的定义和操控

- **定义：**数据类型 * 指针变量名
- `int *p;` (定义了个指针)
- `p = &a` (调用了指针)
- **使用：**可以使用解引用的方式找到指针指向的内存
- `*p` (表示解引用)


```

1  int main()
2  {
3      int a = 10;
4      //创立指针
5      int *p;
6      //记录变量a的地址
7      p = &a;
8      //解引用
9      cout << *p<<endl;
10     *p = 1000; //指针也可以修改内存
11     cout << a;
12
13     return 0;
14 }

```

7.3 指针所占的内存空间

- 在32位操作系统下，指针占用4字节

```

1  int main()
2  {
3      //指针的第二种写法
4      int a = 10;
5      int *p = &a;
6
7      cout << "*p所占的内存为"<<sizeof(int *); //我这里似乎是64位系统，所以输出结果是8
8
9      return 0;
10 }

```

7.4 空指针与野指针

- **空指针**：指向内存中位0位的指针
- **野指针**：初始化指针
- **空指针指向的内存是无法被访问的**
- **野指针**：指针指向非法的内存空间
 - 在没有申请内存的情况使用指针访问这串内存

7.5 const修饰指针

有三种情况：

1. const修饰指针→ **常量指针**
 1. 指针的指向可以更改，但指针指向的值不可以改
- 2.

7.6 数组与指针的关系[原创]

- 对于一个数组而言，我们可以把它理解为一段内存地址 + 一个指针: 其中，指针名就是指向第一个内存地址的指针

```
1 | a          [内存地址1][内存地址2][内存地址3].....
2 | -> 指针名字|->地址1
3 | -----> 地址2
```

重新审视数组定义：

```
1 | int arr[n] --> 数组大小 *1
2 | |   |--> 数组名称    *2
3 | |   |--> 数组类型    *3
```

再来审视一下指针定义

```
1 | int *p = NULL --> 指针指向的变量
2 | |   |-->指针名称
3 | |   |-->指针类型
```

- 那对于一个数组而言，我们不妨把数组名字当作一个指针名，而 [] 当作一个解指针+指针运算的方式

```
1 | *(p + 0) == p[0]
```

💡 Tip

注意: [] 是确实存在的，并且不仅是数组可以使用，但是注意在定义指针数组的时候，`int *p[a]` 这类的操作下[]的含义又变得不同

那对一个二维数组而言，其本质就是一个二重指针，里层的指针指向元素，外层的指针指向里层的指针，两次[]表示运算两次指针+解两次指针

指针数组与数组指针

8 结构体

自定义的数据类型，允许用户储存不同的数据类型

8.1 结构体的定义

语法: `struct 结构体名{结构体成员列表};`

- 有三种创建变量的方式:
 - `struct 结构体名 变量名`
 - `struct 结构体名 变量名 = {成员1, 成员2, .....}`
 - 定义结构体时顺便创建变量

例:

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  // 创建数据类型
5  struct QAQ {
6      string s1;
7      int n1;
8      int n2;
9      // 下面是定义时候定义
10 } k3;
11
12 int main()
13 {
14     // 定义结构体数据 1
15     QAQ k1;
16     k1.s1 = "aaa";
17     // 定义结构体数据 2
18     QAQ k2 = { "114", 5, 1 };
19     k3.s1 = "1919";
20     k3.n1 = 810;
21     return 0;
22 }
```

8.2 结构体数组

定义结构体放入数组方便维护

- **语法:** `struct 结构体名 数组名[元素个数] = {};`

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  // 创建数据类型
5  struct QAQ {
6      string s1;
7      int n1;
8      int n2;
9  } s[10];
10
11 int main()
12 {
13     s[1] = { "QAQ", 1, 2 };
14     cout << s[1].s1;
```

```

15     // 或者
16     QAQ kk[5]; //第二种方法构建
17     cin >> kk[3].s1;
18     cout << kk[3].s1;
19     return 0;
20 }

```

8.3 数组结构体[原创]

- 结构体可以用数组表示，数组当然也可以放在结构体中

`int a[n]` 类型

```

1  struct test1
2  {
3      string s;
4      int a[9];
5  };
6
7  signed main()
8  {
9      //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
10     int t = 1;
11     cin >> t;
12     test1 st[t];
13     for (size_t i = 0; i < t; i++) {
14         cin >> st[i].s;
15         for (size_t j = 0; j < 9; j++) {
16             cin >> st[i].a[j];
17         }
18     }
19     return 0;
20 }

```

注意上述 `int a[n]` 中的 `n` 为一个确定常数

`vector<int>` 类型

```

1  struct test1
2  {
3      string s;
4      vector<int> a;
5  };
6
7  signed main()
8  {
9      //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
10     int t = 1;
11     cin >> t;
12     test1 st[t];
13     for (size_t i = 0; i < t; i++) {
14         cin >> st[i].s;
15         for (size_t j = 0; j < 3; j++) {
16             int p;

```

```

17         cin >> p;
18         st[i].a.push_back(p);
19     }
20 }
21 for (auto &&i : st) {
22     cout << i.s << " ";
23     for (auto &&j : i.a) {
24         cout << j << " ";
25     }
26     cout << endl;
27 }
28 return 0;
29 }

```

Important

在上述构造方法中都没有对结构体里的数组进行初始化，下面是初始化的方法

对 `int a[n]` 而言：

- 直接 结构体里 `int a[n] = {0};` 即可

对 `vector<int>` 而言：

- 采取显式调用即可

```
1 vector<int> a = vector<int>(n,0);
```

- 其中 n 为常数

8.4 结构体的构造函数

结构体构造函数的形式与用法

在 C++ 中，结构体（`struct`）的构造函数和类（`class`）的构造函数没有本质区别。构造函数是特殊的成员函数，用于在创建对象时初始化成员变量。构造函数的名称必须与结构体的名称相同。

构造函数的形式

1. 默认构造函数

不带参数的构造函数，用于初始化成员变量为默认值。

```

1 #include <iostream>
2 #include <string>
3
4 struct Person {
5     std::string name;
6     int age;
7
8     // 默认构造函数
9     Person() : name("unknown"), age(0) {} // 使用初始化列表

```

```
10 };
11
12 int main() {
13     Person p; // 自动调用默认构造函数
14     std::cout << "Name: " << p.name << ", Age: " << p.age << "\n";
15     return 0;
16 }
```

2. 带参数的构造函数

构造函数可以接受参数，用于动态初始化成员变量。

```
1 #include <iostream>
2 #include <string>
3
4 struct Person {
5     std::string name;
6     int age;
7
8     // 带参数的构造函数
9     Person(const std::string& n, int a) : name(n), age(a) {}
10 };
11
12 int main() {
13     Person p("Alice", 25); // 调用带参数的构造函数
14     std::cout << "Name: " << p.name << ", Age: " << p.age << "\n";
15     return 0;
16 }
```

3. 构造函数重载

通过不同的参数列表定义多个构造函数，满足不同的初始化需求。

```
1 #include <iostream>
2 #include <string>
3
4 struct Person {
5     std::string name;
6     int age;
7
8     // 默认构造函数
9     Person() : name("unknown"), age(0) {}
10
11     // 带参数的构造函数
12     Person(const std::string& n, int a) : name(n), age(a) {}
13
14     // 只初始化名字
15     Person(const std::string& n) : name(n), age(18) {} // 默认年龄为18
16 };
17
18 int main() {
19     Person p1; // 默认构造
20     Person p2("Bob", 30); // 带参数
```

```

21 Person p3("Charlie");           // 只提供名字
22
23 std::cout << p1.name << ", " << p1.age << "\n";
24 std::cout << p2.name << ", " << p2.age << "\n";
25 std::cout << p3.name << ", " << p3.age << "\n";
26
27 return 0;
28 }

```

输出

```

1 unknown, 0
2 Bob, 30
3 Charlie, 18

```

4. 使用初始化列表

初始化列表用于直接初始化成员变量，避免在构造函数体内赋值。

```

1 #include <iostream>
2 #include <string>
3
4 struct Person {
5     std::string name;
6     int age;
7
8     // 使用初始化列表
9     Person(const std::string& n, int a) : name(n), age(a) {}
10 };
11
12 int main() {
13     Person p("Diana", 22); // 初始化时直接调用
14     std::cout << "Name: " << p.name << ", Age: " << p.age << "\n";
15     return 0;
16 }

```

初始化列表的优点：

1. 提高效率：避免默认构造后再赋值。
2. 支持 `const` 和引用类型的初始化。

特殊形式

1. 委托构造函数（C++11 起支持）

一个构造函数可以委托给另一个构造函数以复用初始化逻辑。

```

1 #include <iostream>
2 #include <string>
3
4 struct Person {
5     std::string name;

```

```

6     int age;
7
8     // 默认构造函数
9     Person() : Person("unknown", 0) {} // 委托到另一个构造函数
10
11    // 带参数的构造函数
12    Person(const std::string& n, int a) : name(n), age(a) {}
13 };
14
15 int main() {
16     Person p1;           // 默认构造
17     Person p2("Eve", 28); // 带参数构造
18
19     std::cout << p1.name << ", " << p1.age << "\n";
20     std::cout << p2.name << ", " << p2.age << "\n";
21
22     return 0;
23 }

```

2. 删除的构造函数 (C++11 起支持)

可以显式删除某些构造函数，防止被意外调用。

```

1 #include <iostream>
2
3 struct Person {
4     int age;
5
6     // 禁止隐式转换或默认构造
7     Person() = delete; // 删除默认构造函数
8     Person(int a) : age(a) {}
9 };
10
11 int main() {
12     // Person p1; // 编译错误：默认构造函数被删除
13     Person p2(30); // 必须传入参数
14     std::cout << "Age: " << p2.age << "\n";
15     return 0;
16 }

```

构造函数的应用场景

1. 初始化成员变量
 - 为结构体或类的成员变量赋初始值，避免对象处于未定义状态。
2. 动态控制初始化
 - 可以根据参数灵活地初始化不同状态的对象。
3. 封装复杂逻辑
 - 在构造函数中封装一些初始化逻辑，简化外部代码。
4. 提高代码安全性

- 使用委托构造或删除某些构造函数可以避免意外使用。

总结

- 构造函数是结构体中用来初始化成员变量的核心工具。
- 通过**默认构造**、**参数化构造**、**初始化列表**等形式，可以满足多样化的初始化需求。
- 推荐使用 **初始化列表**，特别是在初始化复杂类型（如 `std::vector`、引用、`const` 变量）时，效率更高。

虽然一般构造都用的是显式调用就是了（

链表

- 什么是链表：
 - 链表是一种用于存储数据的数据结构，通过如链条一般的指针来连接元素。它的特点是插入与删除数据十分方便，但寻找与读取数据的表现欠佳。
- 链表的优势：对数据的处理：插入，删除 $\rightarrow O(1)$ 但也因为这样，寻找、读取数据的效率不如数组高，在随机访问数据中的操作次数是 $O(n)$

与数组相反的是，数组在随机访问数据下时间复杂度为 $O(1)$ ，但插入删除数据为 $O(n)$

单向链表与双向链表的创建

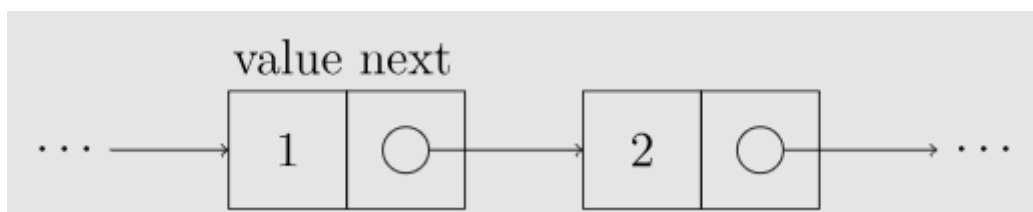
- **单向链表**：单向链表包括两大数据类型，即数据域和指针域

```
1 struct Node
2 {
3     int value;
4     Node *next; // 定义了一个指向该结构体(链表单位)的一个指针
5     Node(int val) : value(val), next(nullptr){}
6 };
```

💡 Tip

这个结构体函数做了什么？

- 初始化了一个新创建的结点，这个结点传入的参数默认赋值给了value，同时将结构体指针域默认赋值为了nullptr,表示其暂时不指向任何其他结点

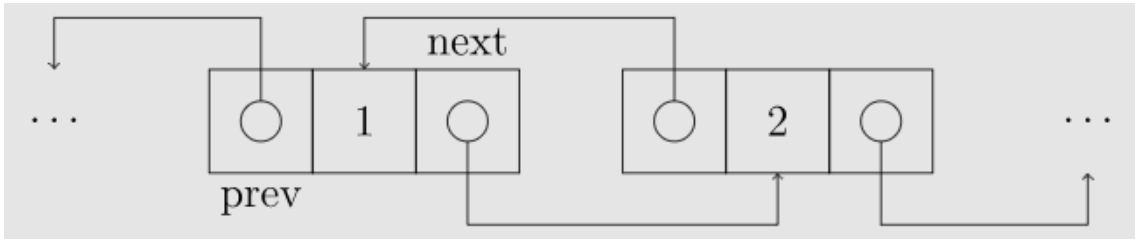


- **双向链表**
- **双向链表**：也同样是数据域+指针域，但不同的是，指针域有左右(或上下)之分，用来链接上一个结点，当前结点，下一个结点

```

1 struct Node {
2     int value;
3     Node *left;
4     Node *right;
5     Node(int val) : value(val) , left(nullptr) , right(nullptr){}
6 };

```



向链表里写入与删去数据

流程大致如下

- 初始化待插入的数据 `node`;
- 将 `node` 的 `next` 指针指向 `p` 的下一个结点;
- 将 `p` 的 `next` 指针指向 `node`。

但在这之前，我们要创建一个链表头

- `Node *head = nullptr`

我们的操作都会在这个头链表上开始

在头部插入数据

时间复杂度 $O(1)$

```

1 void insert_head_node(Node *newnode, Node *&head)
2 {
3     newnode->next = head;
4     head = newnode;
5 }

```

在某一个pos值上插入元素

```

1 void insert_pos_node(Node *newnode, int pos, Node *&head)
2 {
3     if (pos == 0) {
4         newnode->next = head;
5         head = newnode;
6         return;
7     }
8     Node *current = head;
9     int currentposition = 0;
10    while (current != nullptr && currentposition < pos - 1) {
11        current = current->next;
12        currentposition++;
13    }
14    newnode->next = current->next;
15    current->next = newnode;

```

在尾部插入元素

在单向链表尾部插入元素，时间复杂的 $O(n)$

```

1 void insert_end_node(Node *Newnode, Node *&p)
2 {
3     // Node *Newnode = new Node(i);
4     if (p == nullptr) {
5         p = Newnode;
6         return;
7     }
8     Node *current = p;
9     while (current->next != nullptr) {
10         current = current->next;
11     }
12     current->next = Newnode;
13 }
```

删除某一结点

- 删除某一特定值的结点

```

1 void delnode(int i, Node *&p)
2 {
3     //链表为空
4     if (p == nullptr) return;
5     //头结点为目标值
6     while (p != nullptr && p->val == i) {
7         Node *temp = p;
8         p = p->next;
9         delete temp;
10        t--;
11    }
12    //中间或尾部结点为目标
13    Node *current = p;
14    while (current != nullptr && current->next != nullptr) {
15        if (current->next->val == i) {
16            Node *temp = current->next;
17            current->next = current->next->next;
18            delete temp;
19            t--;
20        }
21        else {
22            current = current->next;
23        }
24    }
25 }
```

- 删除某一位置的结点

```

1 void posdel(int pos, Node *&head)
```

```

2  {
3      if (head == nullptr || pos < 0) return;
4      if (pos == 0) {
5          Node *temp = head;
6          head = head->next;
7          delete temp;
8          return;
9      }
10     Node *current = head;
11     for (int i = 0; i < pos && current != nullptr; i++) {
12         current = current->next;
13     }
14     if (current == nullptr || current->next == nullptr) return;
15     Node *temp = current->next;
16     current->next = current->next->next;
17     delete temp;
18 }

```

9 类

- 在面向对象编程（OOP）中，对象就是类的实例，也就是变量

```

1  class Class_Name
2  {
3  public:
4      string str;
5      int adds;
6      void add_age(int add_s)
7      {
8          //.....//
9      } // 内联函数
10     void getval();
11 private:
12     int age;
13 };

```

对内联函数，也可以使用 inline 来在类的外部写

```

1  inline

```